# Remove Kth Node From End

AlgoExpert · Medium · Linked Lists

# The problem

- Remove in place (list length >= 2, k valid).
- If the head is removed, the head node takes its successor's value so the reference stays valid.

# Intuition

- Keep a gap of k between two pointers so the leading one hits the end when the trailing one is just before the target.

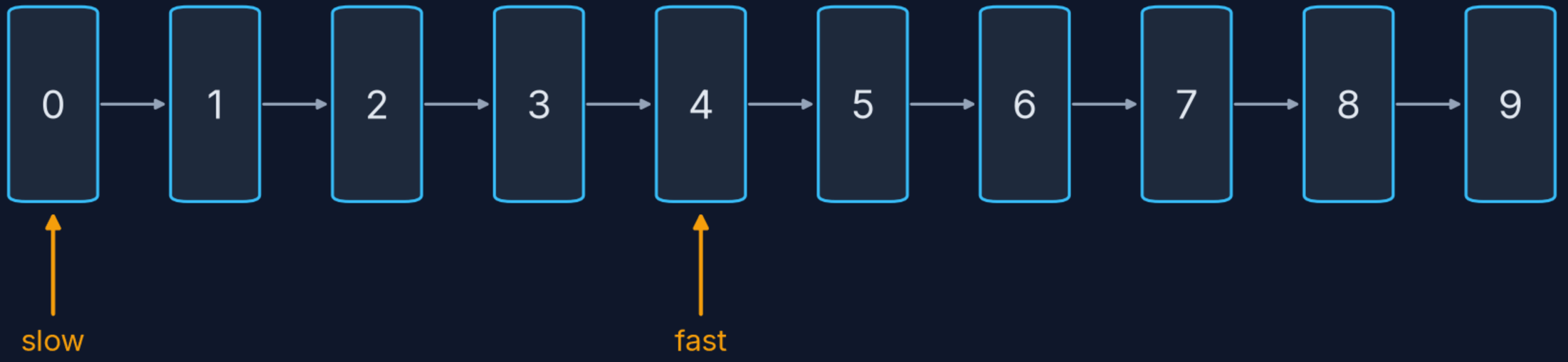
# Brute force vs optimal

- Two passes: count length, then walk to n − k
- One pass with a k-gap between two pointers
- Both O(n) time, O(1) space

# Approach

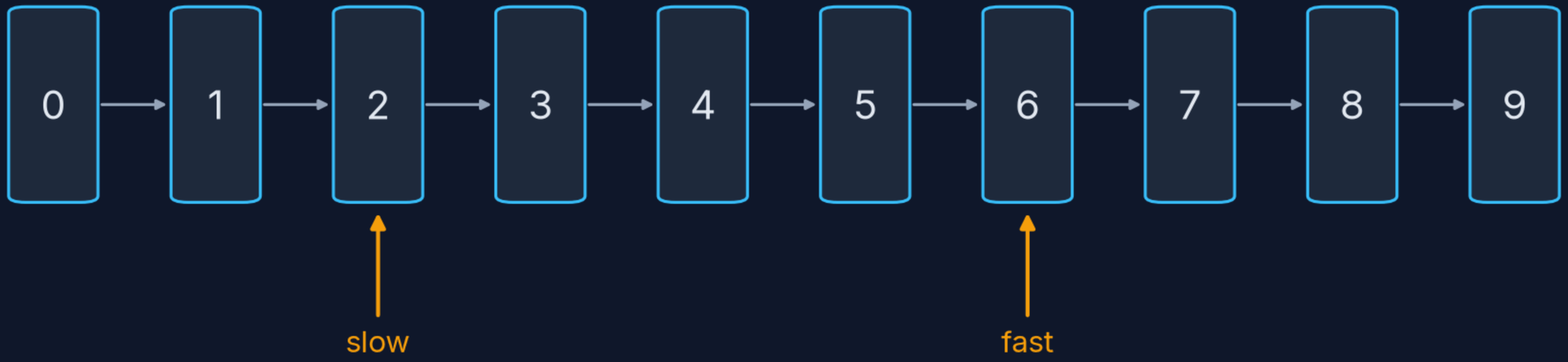
- 1. Advance the lead k steps
- 2. Move both until the lead ends
- 3. Unlink the target, handling the head

# Worked example



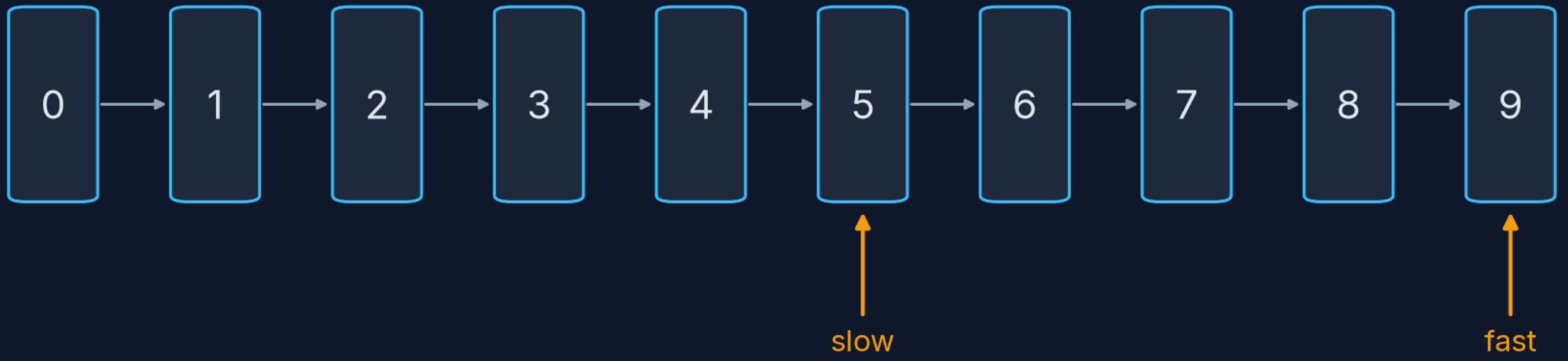

Move fast 4 steps ahead

# Worked example



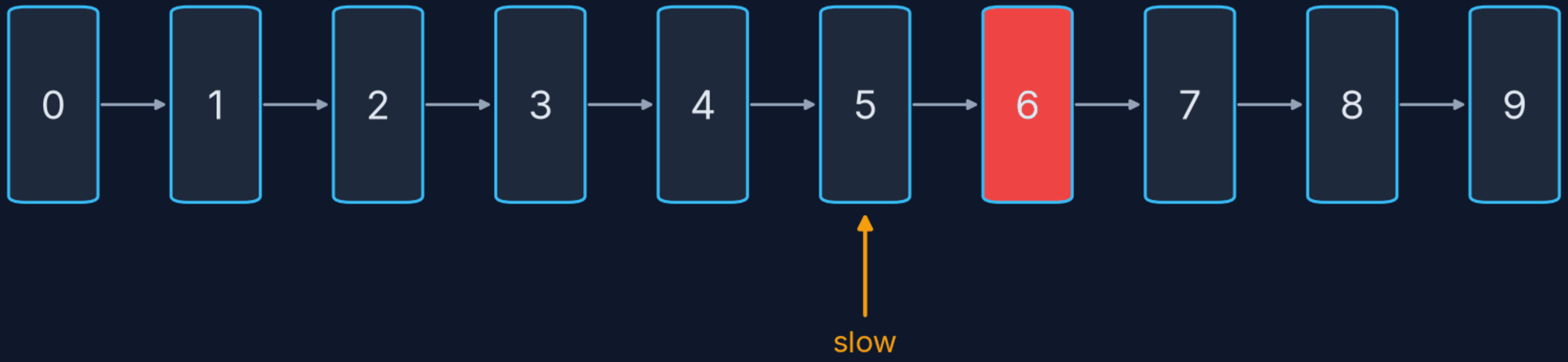

Advance both: slow at 2

# Worked example




Advance both: slow at 5

# Worked example




slow.next = slow.next.next removes 6

# Java solution

```java
public class RemoveKthNodeFromEnd {
    public static void removeKthNodeFromEnd(LinkedListNode head, int k) {
        LinkedListNode fast = head, slow = head;
        for (int i = 0; i < k; i++) fast = fast.next;
        if (fast == null) { head.value = head.next.value; head.next = head.next.next; return; }
        while (fast.next != null) { fast = fast.next; slow = slow.next; }
        slow.next = slow.next.next;
    }
}
```

# Complexity

- Time: O(n)
- Space: O(1)

# Edge cases & pitfalls

- Removing the head (k = length): copy next into head
- K is always valid here
- Stop fast at the last node, not null

# Interview tips & follow-ups

- Dummy node simplifies head removal
- Follow-up: return the new head instead

# Recap

- Keep a gap of k between two pointers so the leading one hits the end when the trailing one is just before the target.
- Complexity: O(n)